

服务器与密钥运维清单

原清单内容整理版 | 真实密码与密钥已替换为占位符

一、服务器基础信息

项目	原清单记录
云服务	腾讯云轻量应用服务器
公网 IP	43.129.246.127
操作系统	Ubuntu 24.04 LTS
SSH 用户名	ubuntu
SSH 密码	<已移除，请从密码管理器获取>
SSH 登录命令	ssh ubuntu@43.129.246.127
防火墙开放端口	4000/TCP，来源 0.0.0.0/0

密码保存

SSH 密码使用强密码，并保存在密码管理器中。

二、LiteLLM 代理部署路径与配置文件

项目	路径或内容
项目目录	/home/ubuntu/litellm-proxy
docker-compose.yml	定义 litellm 和 db 两个服务；映射 4000 端口；挂载 ./config.yaml

项目	路径或内容
config.yaml	配置 model_list、Kimi 模型、API Key 与 Moonshot API Base
.env	保存 MOONSHOT_API_KEY、LITELLM_MASTER_KEY、DB_PASSWORD
postgres_data	数据库持久化目录（容器卷）

config.yaml 原清单结构

```
model_list:
- model_name: kimi-k2.5
  litellm_params:
    model: moonshot/kimi-k2.5
    api_key: <MOONSHOT_API_KEY>
    api_base: https://api.moonshot.cn/v1
```

三、关键密钥与密码清单

名称	作用	整理后记录
Kimi API Key	调用 Kimi 官方接口（上游）	<已移除>
LiteLLM 主密钥	管理代理、生成虚拟密钥、访问管理接口	<已移除>
PostgreSQL 数据库密码	连接数据库	<请从密码管理器获取>
服务器 SSH 密码	登录服务器	<请从密码管理器获取>

敏感信息

API Key、主密钥、数据库密码和 SSH 密码不在此文档中保存。

四、服务器日常管理命令

登录服务器

```
ssh ubuntu@43.129.246.127
```

进入 LiteLLM 目录

```
cd ~/litellm-proxy
```

查看服务状态

```
sudo docker compose ps  
sudo docker compose logs litellm --tail 50
```

重启服务（修改配置后）

```
sudo docker compose down  
sudo docker compose up -d
```

查看容器资源占用

```
sudo docker stats --no-stream  
free -h
```

进入 LiteLLM 容器内部

```
sudo docker exec -it litellm-proxy-litellm-1 bash
```

查看数据库日志

```
sudo docker compose logs db --tail 30
```

五、为用户生成虚拟密钥

每次为一个新用户生成独立虚拟密钥。

命令模板

```
curl -X POST "http://localhost:4000/key/generate" \
-H "Authorization: Bearer $LITELLM_MASTER_KEY" \
-H "Content-Type: application/json" \
-d '{"alias": "用户名", "models": ["kimi-k2.5"]}'
```

示例：用户名为 zhangsan

```
curl -X POST "http://localhost:4000/key/generate" \
-H "Authorization: Bearer $LITELLM_MASTER_KEY" \
-H "Content-Type: application/json" \
-d '{"alias": "zhangsan", "models": ["kimi-k2.5"]}'
```

返回结果：提取 key 字段的值，作为分发给该用户的虚拟密钥。

六、管理虚拟密钥

查看所有虚拟密钥

```
curl -X GET "http://localhost:4000/key/list" \
-H "Authorization: Bearer $LITELLM_MASTER_KEY"
```

查看某个虚拟密钥

```
curl -X GET "http://localhost:4000/key/info?key=sk-xxx" \
-H "Authorization: Bearer $LITELLM_MASTER_KEY"
```

删除某个虚拟密钥

```
curl -X POST "http://localhost:4000/key/delete" \
-H "Authorization: Bearer $LITELLM_MASTER_KEY" \
-H "Content-Type: application/json" \
-d '{"keys": ["sk-xxx"]}'
```

七、用户分发信息与配置

项目	原清单内容
LiteLLM 网关地址	http://43.129.246.127:4000
用户虚拟密钥	<用户专属虚拟密钥>
可用模型名称	kimi-k2.5

用户配置步骤

顺序	操作
1	克隆仓库
2	复制 config.example.json 为 config.json
3	将 apiKey 字段改为用户虚拟密钥
4	确保 baseUrl 指向 LiteLLM 网关地址
5	运行 docker-compose up -d

原清单中的其他配置记录

文件或目录	记录内容
supabaseconfig.json	将 apiKey 字段替换为 <原值已移除>
config.json	将 apiKey 字段替换为 <用户专属虚拟密钥>
secrets/	放入 double-scholar-487115-b1-075776a1689b.json

八、生成 DB_PROXY_API_KEY

OpenSSL

```
openssl rand -hex 32
```

输出为 32 字节随机数据，即 64 个十六进制字符，可作为 DB_PROXY_API_KEY。

Windows PowerShell

```
$bytes = [byte[]]::new(32)
[System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$key = [Convert]::ToHexString($bytes).ToLower()
Write-Output $key
```

Python

```
import secrets
print(secrets.token_hex(32))
```

Linux / macOS

```
cat /dev/urandom | tr -dc 'a-f0-9' | fold -w 64 | head -n 1
```

九、Docker 多架构发布记录

执行命令

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File ".\scripts\publish-multiarch.ps1"
```

项目	日志记录
Buildx Builder	claw-multiarch
Driver	docker-container
目标平台	linux/amd64、linux/arm64
构建阶段	45/45 步骤完成
推送阶段	上传 Docker Hub 时出现 broken pipe / closed network connection
最终结果	获取 Docker Hub OAuth token 时 EOF；后端多平台发布失败